

Machine Learning Algorithms Laboratory

Experiment 2

Email Spam/Ham Classification using Naïve Bayes and KNN

Name: Jayasri M
Register No.: 3122247001025

Aim

To classify emails as spam or ham using Naïve Bayes and K-Nearest Neighbors (KNN) algorithms and compare their performance.

Objective

- Implement Naïve Bayes and KNN classifiers.
- Compare Gaussian, Multinomial and Bernoulli Naïve Bayes.
- Study the effect of different values of k in KNN.
- Tune the KNN model using GridSearchCV and RandomizedSearchCV.
- Compare the performance of the classifiers.

Theory

Naïve Bayes is a supervised learning algorithm based on Bayes' theorem. It assumes that the features are independent of each other. It is widely used for text classification problems like spam detection.

K-Nearest Neighbors (KNN) is a supervised learning algorithm that classifies a new sample by looking at its nearest neighbours. The prediction depends on the value of k , which represents the number of neighbours considered.

Dataset

The Spambase dataset from Kaggle was used in this experiment. It contains email samples with numerical features and a target column that classifies each email as spam or ham.

EDA

The following steps were performed during exploratory data analysis.

- Loaded the dataset.
- Checked the shape and information of the dataset.
- Verified missing values.
- Displayed summary statistics.
- Plotted class distribution.
- Generated histograms, boxplots and correlation heatmap.
- Normalized the features before training the models.

```

import time

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.model_selection import GridSearchCV
from sklearn.model_selection import RandomizedSearchCV
from sklearn.model_selection import cross_val_score
from sklearn.preprocessing import MinMaxScaler

from sklearn.naive_bayes import GaussianNB
from sklearn.naive_bayes import MultinomialNB
from sklearn.naive_bayes import BernoulliNB
from sklearn.neighbors import KNeighborsClassifier

from sklearn.metrics import accuracy_score
from sklearn.metrics import precision_score
from sklearn.metrics import recall_score
from sklearn.metrics import f1_score
from sklearn.metrics import roc_auc_score
from sklearn.metrics import confusion_matrix
from sklearn.metrics import ConfusionMatrixDisplay
from sklearn.metrics import RocCurveDisplay
from sklearn.metrics import PrecisionRecallDisplay
df = pd.read_csv("spambase_csv.csv")
print("Dataset Loaded Successfully")
Dataset Loaded Successfully
df = pd.read_csv("spambase_csv.csv")
print("Dataset Loaded Successfully")
Dataset Loaded Successfully
df.head()

```

	word_freq_make	word_freq_address	word_freq_all	word_freq_3d	\
0	0.00	0.64	0.64	0.0	
1	0.21	0.28	0.50	0.0	
2	0.06	0.00	0.71	0.0	
3	0.00	0.00	0.00	0.0	
4	0.00	0.00	0.00	0.0	
	word_freq_out	word_freq_over	word_freq_remove	word_freq_internet	

```

\
0      0.32      0.00      0.00      0.00
1      0.14      0.28      0.21      0.07
2      1.23      0.19      0.19      0.12
3      0.63      0.00      0.31      0.63
4      0.63      0.00      0.31      0.63

word_freq_order word_freq_mail ... char_freq_%3B char_freq_%28
0.00      0.00 ...      0.00      0.000

\
0
1      0.00      0.94 ...      0.00      0.132
2      0.64      0.25 ...      0.01      0.143
3      0.31      0.63 ...      0.00      0.137 4
0.31      0.63 ...      0.00      0.135
char_freq_%5B char_freq_%21 char_freq_%24 char_freq_%23 \
0      0.0      0.778      0.000      0.000
1      0.0      0.372      0.180      0.048
2      0.0      0.276      0.184      0.010
3      0.0      0.137      0.000      0.000
4      0.0      0.135      0.000      0.000
capital_run_length_average capital_run_length_longest \
0      3.756      6 1
1      5.114      10 1
2      9.821      48 5
3      3.537      4 0
4      3.537      4 0
capital_run_length_total class
0      278      1
1      1028
      1
2      2259
      1
3      191      1
4      191      1
[5 rows x 58 columns]
print("Shape :", df.shape) print("\nColumns")

print(df.columns)

print("\nData Types") print(df.dtypes)

print("\nInformation")
df.info() Shape :
(4601, 58)

```

Columns

```
Index(['word_freq_make', 'word_freq_address', 'word_freq_all',  
      'word_freq_3d',  
      'word_freq_our', 'word_freq_over', 'word_freq_remove',  
      'word_freq_internet', 'word_freq_order', 'word_freq_mail',  
      'word_freq_receive', 'word_freq_will', 'word_freq_people',  
      'word_freq_report', 'word_freq_addresses', 'word_freq_free',  
      'word_freq_business', 'word_freq_email', 'word_freq_you',  
      'word_freq_credit', 'word_freq_your', 'word_freq_font',  
      'word_freq_000',  
      'word_freq_money', 'word_freq_hp', 'word_freq_hpl',  
      'word_freq_george',  
      'word_freq_650', 'word_freq_lab', 'word_freq_labs',  
      'word_freq_telnet',  
      'word_freq_857', 'word_freq_data', 'word_freq_415',  
      'word_freq_85',  
      'word_freq_technology', 'word_freq_1999', 'word_freq_parts',  
      'word_freq_pm', 'word_freq_direct', 'word_freq_cs',  
      'word_freq_meeting',  
      'word_freq_original', 'word_freq_project', 'word_freq_re',  
      'word_freq_edu', 'word_freq_table', 'word_freq_conference',  
      'char_freq_%3B', 'char_freq_%28', 'char_freq_%5B', 'char_freq_  
%21',  
      'char_freq_%24', 'char_freq_%23', 'capital_run_length_average',  
      'capital_run_length_longest', 'capital_run_length_total',  
      'class'],  
      dtype='str')
```

Data Types

word_freq_make	float64	
word_freq_address	float64	
word_freq_all	float64	word_freq_3d
float64 word_freq_our		float64
word_freq_over	float64	
word_freq_remove	float64	
word_freq_internet	float64	
word_freq_order	float64	
word_freq_mail	float64	
word_freq_receive	float64	
word_freq_will	float64	
word_freq_people	float64	
word_freq_report	float64	
word_freq_addresses	float64	
word_freq_free	float64	
word_freq_business	float64	
word_freq_email	float64	
word_freq_you	float64	
word_freq_credit	float64	
word_freq_your	float64	
word_freq_font	float64	
word_freq_000	float64	
word_freq_money	float64	word_freq_hp
float64 word_freq_hpl		float64
word_freq_george	float64	
word_freq_650	float64	
word_freq_lab	float64	
word_freq_labs	float64	
word_freq_telnet	float64	

```

word_freq_857 float64
word_freq_data float64
word_freq_415 float64 word_freq_85
float64 word_freq_technology float64
word_freq_1999 float64
word_freq_parts float64 word_freq_pm
float64 word_freq_direct float64
word_freq_cs float64
word_freq_meeting float64
word_freq_original float64
word_freq_project float64 word_freq_re
float64 word_freq_edu float64
word_freq_table float64
word_freq_conference float64
char_freq_%3B float64
char_freq_%28 float64
char_freq_%5B float64
char_freq_%21 float64
char_freq_%24 float64
char_freq_%23 float64
capital_run_length_average float64
capital_run_length_longest int64
capital_run_length_total int64 class
int64

```

dtype: object

Information

<class 'pandas.DataFrame'>

RangeIndex: 4601 entries, 0 to 4600 Data

columns (total 58 columns):

#	Column	Non-Null Count	Dtype
0	word_freq_make	4601 non-null	float64
1	word_freq_address	4601 non-null	float64
2	word_freq_all	4601 non-null	float64
3	word_freq_3d	4601 non-null	float64
4	word_freq_our	4601 non-null	float64
5	word_freq_over	4601 non-null	float64
6	word_freq_remove	4601 non-null	float64
7	word_freq_internet	4601 non-null	float64
8	word_freq_order	4601 non-null	float64
9	word_freq_mail	4601 non-null	float64
10	word_freq_receive	4601 non-null	float64
11	word_freq_will	4601 non-null	float64
12	word_freq_people	4601 non-null	float64
13	word_freq_report	4601 non-null	float64
14	word_freq_addresses	4601 non-null	float64
15	word_freq_free	4601 non-null	float64
16	word_freq_business	4601 non-null	float64
17	word_freq_email	4601 non-null	float64
18	word_freq_you	4601 non-null	float64
19	word_freq_credit	4601 non-null	float64
20	word_freq_your	4601 non-null	float64
21	word_freq_font	4601 non-null	float64
22	word_freq_000	4601 non-null	float64
23	word_freq_money	4601 non-null	float64
24	word_freq_hp	4601 non-null	float64
25	word_freq_hpl	4601 non-null	float64

26	word_freq_george	4601	non-null	float64
27	word_freq_650	4601	non-null	float64
28	word_freq_lab	4601	non-null	float64
29	word_freq_labs	4601	non-null	float64
30	word_freq_telnet	4601	non-null	float64
31	word_freq_857	4601	non-null	float64
32	word_freq_data	4601	non-null	float64
33	word_freq_415	4601	non-null	float64
34	word_freq_85	4601	non-null	float64
35	word_freq_technology	4601	non-null	float64
36	word_freq_1999	4601	non-null	float64
37	word_freq_parts	4601	non-null	float64
38	word_freq_pm	4601	non-null	float64
39	word_freq_direct	4601	non-null	float64
40	word_freq_cs	4601	non-null	float64
41	word_freq_meeting	4601	non-null	float64
42	word_freq_original	4601	non-null	float64

```

43    word_freq_project      4601 non-null    float64
44    word_freq_re          4601 non-null    float64
45    word_freq_edu         4601 non-null    float64
46    word_freq_table       4601 non-null    float64
47    word_freq_conference  4601 non-null    float64
48    char_freq_%3B         4601 non-null    float64
49    char_freq_%28         4601 non-null    float64
50    char_freq_%5B         4601 non-null    float64
51    char_freq_%21         4601 non-null    float64
52    char_freq_%24         4601 non-null    float64
53    char_freq_%23         4601 non-null    float64
54    capital_run_length_average 4601 non-null    float64
55    capital_run_length_longest 4601 non-null    int64
56    capital_run_length_total 4601 non-null    int64    57    class
4601 non-null    int64
dtypes: float64(55), int64(3)
memory usage: 2.0 MB
df.describe()

```

	word_freq_make	word_freq_address	word_freq_all	word_freq_3d
count	4601.000000	4601.000000	4601.000000	4601.000000

\

mean	0.104553	0.213015	0.280656	0.065425
std	0.305358	1.290575	0.504143	1.395151
min	0.000000	0.000000	0.000000	0.000000
25%	0.000000	0.000000	0.000000	0.000000
50%	0.000000	0.000000	0.000000	0.000000
75%	0.000000	0.000000	0.420000	0.000000
max	4.540000	14.280000	5.100000	42.810000

	word_freq_our	word_freq_over	word_freq_remove
count	4601.000000	4601.000000	4601.000000
mean	0.312223	0.095901	0.114208
std	0.672513	0.273824	0.391441
min	0.000000	0.000000	0.000000
25%	0.000000	0.000000	0.000000
50%	0.000000	0.000000	0.000000
75%	0.380000	0.000000	0.000000
max	10.000000	5.880000	7.270000

	word_freq_order	word_freq_mail	...	char_freq_%3B	char_freq_
%28 \					
count	4601.000000	4601.000000	...	4601.000000	
4601.000000					
mean	0.090067	0.239413	...	0.038575	
0.139030					
std	0.278616	0.644755	...	0.243471	
0.270355					
min	0.000000	0.000000	...	0.000000	
0.000000					
25%	0.000000	0.000000	...	0.000000	
0.000000					
50%	0.000000	0.000000	...	0.000000	
0.065000					
75%	0.000000	0.160000	...	0.000000	
0.188000					
max	5.260000	18.180000	...	4.385000	
9.752000					

	char_freq_%5B	char_freq_%21	char_freq_%24	char_freq_%23	\
count	4601.000000	4601.000000	4601.000000	4601.000000	
mean	0.016976	0.269071	0.075811	0.044238	
std	0.109394	0.815672	0.245882	0.429342	
min	0.000000	0.000000	0.000000	0.000000	
25%	0.000000	0.000000	0.000000	0.000000	
50%	0.000000	0.000000	0.000000	0.000000	75%
0.000000	0.315000	0.052000	0.000000	max	
4.081000	32.478000	6.003000	19.829000		

	capital_run_length_average	capital_run_length_longest	\
count	4601.000000	4601.000000	mean
5.191515	52.172789		std
31.729449	194.891310		min
1.000000	1.000000		25%
1.588000	6.000000		
50%	2.276000		15.000000
75%	3.706000		43.000000
max	1102.500000		9989.000000

	capital_run_length_total	class	
count	4601.000000	4601.000000	mean
283.289285	0.394045		std
606.347851	0.488698		

```
min          1.000000    0.000000
25%          35.000000    0.000000
50%          95.000000    0.000000
75%         266.000000    1.000000
max        15841.000000    1.000000
```

```
[8 rows x 58 columns]
```

```
print(df.isnull().sum())
```

```
word_freq_make          0
word_freq_address       0
word_freq_all           0
word_freq_3d            0
word_freq_our           0
word_freq_over          0
word_freq_remove        0
word_freq_internet      0
word_freq_order         0
word_freq_mail          0
word_freq_receive       0
word_freq_will          0
word_freq_people        0
word_freq_report        0
word_freq_addresses     0
word_freq_free          0
word_freq_business      0
word_freq_email         0
word_freq_you           0
word_freq_credit        0
word_freq_your          0
word_freq_font          0
word_freq_000           0
word_freq_money         0
word_freq_hp            0
word_freq_hpl           0
word_freq_george        0
word_freq_650           0
word_freq_lab           0
word_freq_labs          0
word_freq_telnet        0
word_freq_857           0
word_freq_data          0
word_freq_415           0
word_freq_85            0
word_freq_technology    0
word_freq_1999          0
word_freq_parts         0
word_freq_pm            0
word_freq_direct        0
```

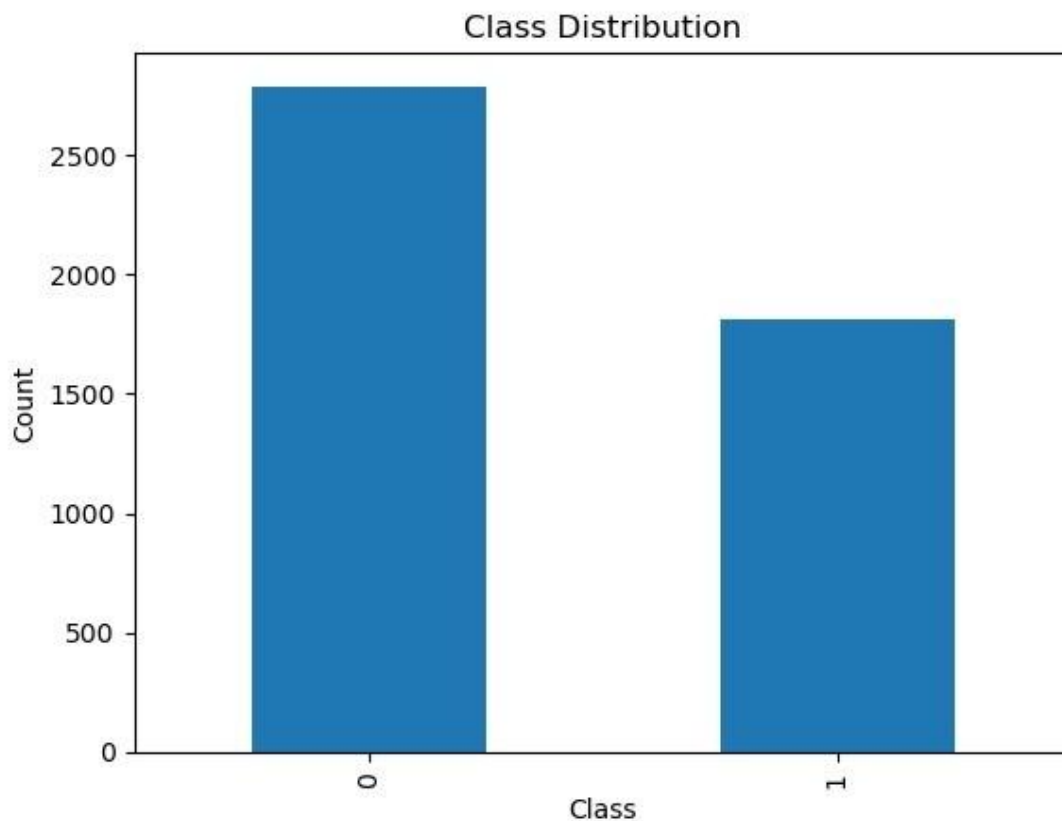
```

word_freq_cs 0
word_freq_meeting 0
word_freq_original 0
word_freq_project 0
word_freq_re 0
word_freq_edu 0
word_freq_table 0
word_freq_conference 0
char_freq_%3B 0
char_freq_%28 0
char_freq_%5B 0
char_freq_%21 0
char_freq_%24 0
char_freq_%23 0
capital_run_length_average 0
capital_run_length_longest 0
capital_run_length_total 0
class 0
dtype: int64

print("Duplicate Rows :", df.duplicated().sum())
Duplicate Rows : 391
df["class"].value_counts().plot(kind="bar")

plt.title("Class Distribution")
plt.xlabel("Class")
plt.ylabel("Count") plt.show()

```



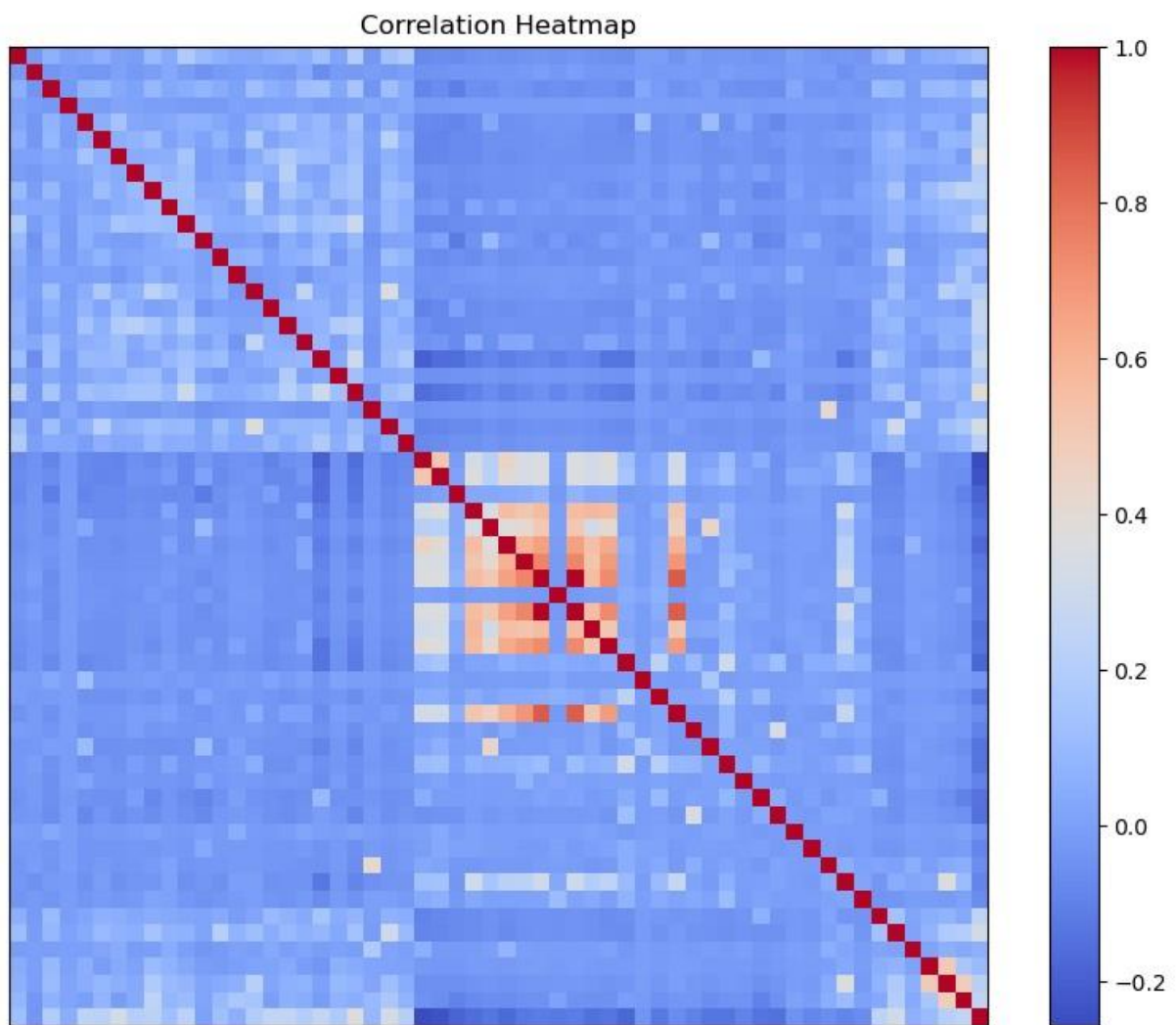
```
plt.figure(figsize=(10,8))

corr = df.corr()

plt.imshow(corr, cmap="coolwarm") plt.colorbar()

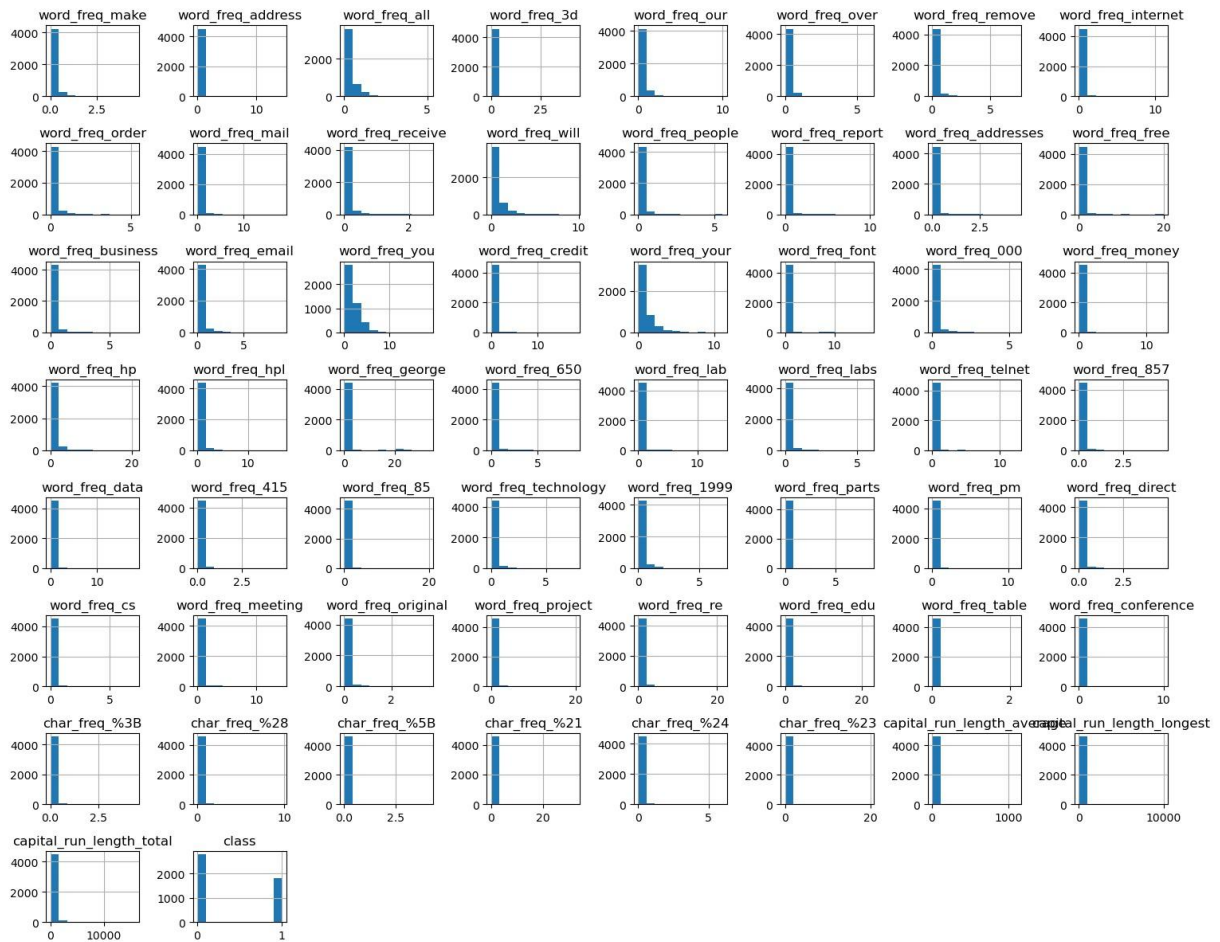
plt.xticks([])
plt.yticks([])

plt.title("Correlation Heatmap")
plt.show()
```



```
df.hist(figsize=(15,12))

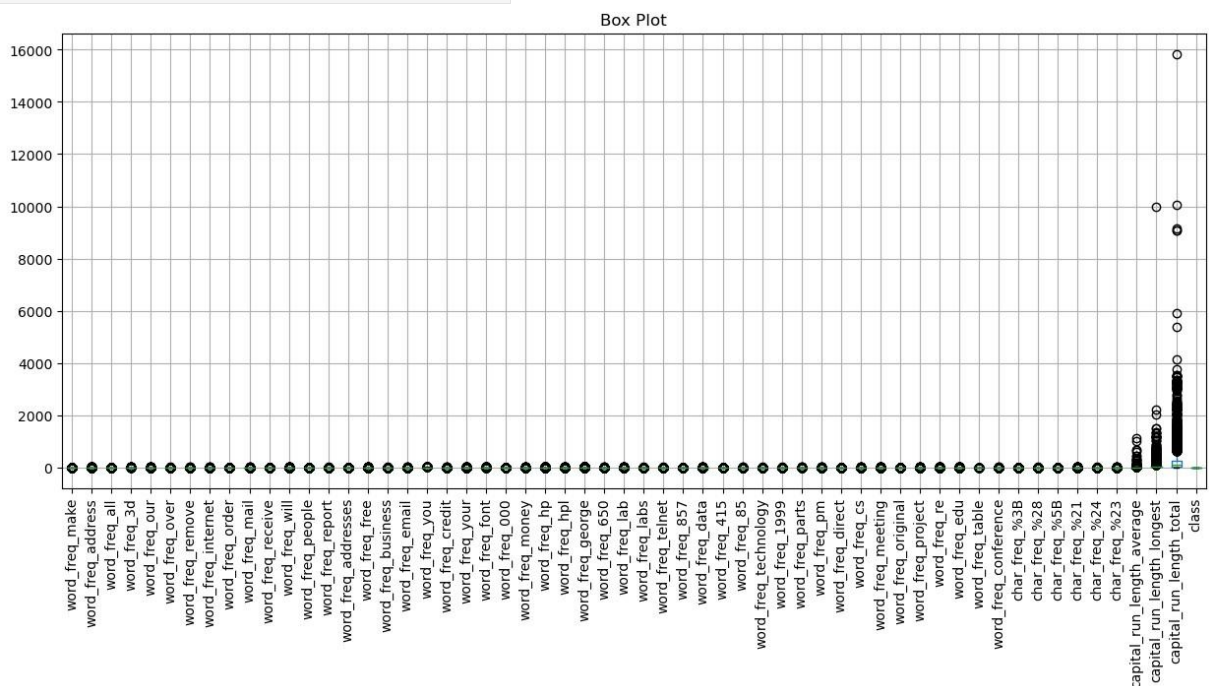
plt.tight_layout() plt.show()
```



```
plt.figure(figsize=(15,6))

df.boxplot(rot=90)

plt.title("Box Plot") plt.show()
```



```
X = df.drop("class", axis=1)
y = df["class"]
scaler = MinMaxScaler()
X = scaler.fit_transform(X)
print("Features Normalized")
Features Normalized
X_train, X_test, y_train, y_test = train_test_split(X, y,
    test_size=0.2,
    random_state=42
)

print("Training Samples :", len(X_train))
print("Testing Samples :", len(X_test))

Training Samples : 3680
Testing Samples : 921
gnb = GaussianNB()
start = time.time()
gnb.fit(X_train, y_train)
```

```
train_time_gnb = time.time() - start
start = time.time()
y_pred_gnb = gnb.predict(X_test)
prediction_time_gnb = time.time() - start
mnb = MultinomialNB() start = time.time()
mnb.fit(X_train, y_train)
train_time_mnb = time.time() - start
start = time.time()
y_pred_mnb = mnb.predict(X_test)
prediction_time_mnb = time.time() - start
bnb = BernoulliNB() start = time.time()
bnb.fit(X_train, y_train)
train_time_bnb = time.time() - start
start = time.time()
y_pred_bnb = bnb.predict(X_test)
prediction_time_bnb = time.time() - start
knn = KNeighborsClassifier(n_neighbors=5)
start = time.time() knn.fit(X_train,
y_train)
train_time_knn = time.time() - start
start = time.time()
y_pred_knn = knn.predict(X_test) prediction_time_knn
= time.time() - start
```

```

def evaluate(y_true, y_pred):

    accuracy = accuracy_score(y_true, y_pred)
    precision = precision_score(y_true, y_pred)    recall
    = recall_score(y_true, y_pred)
    f1 = f1_score(y_true, y_pred)    roc =
    roc_auc_score(y_true, y_pred)    return
    accuracy, precision, recall, f1, roc

g = evaluate(y_test, y_pred_gnb)
m = evaluate(y_test, y_pred_mnb)
b = evaluate(y_test, y_pred_bnb)

nb_results = pd.DataFrame({
    "Metric": ["Accuracy", "Precision", "Recall", "F1", "ROC-AUC"],
    "Gaussian": g,
    "Multinomial": m,
    "Bernoulli": b
})
nb_results

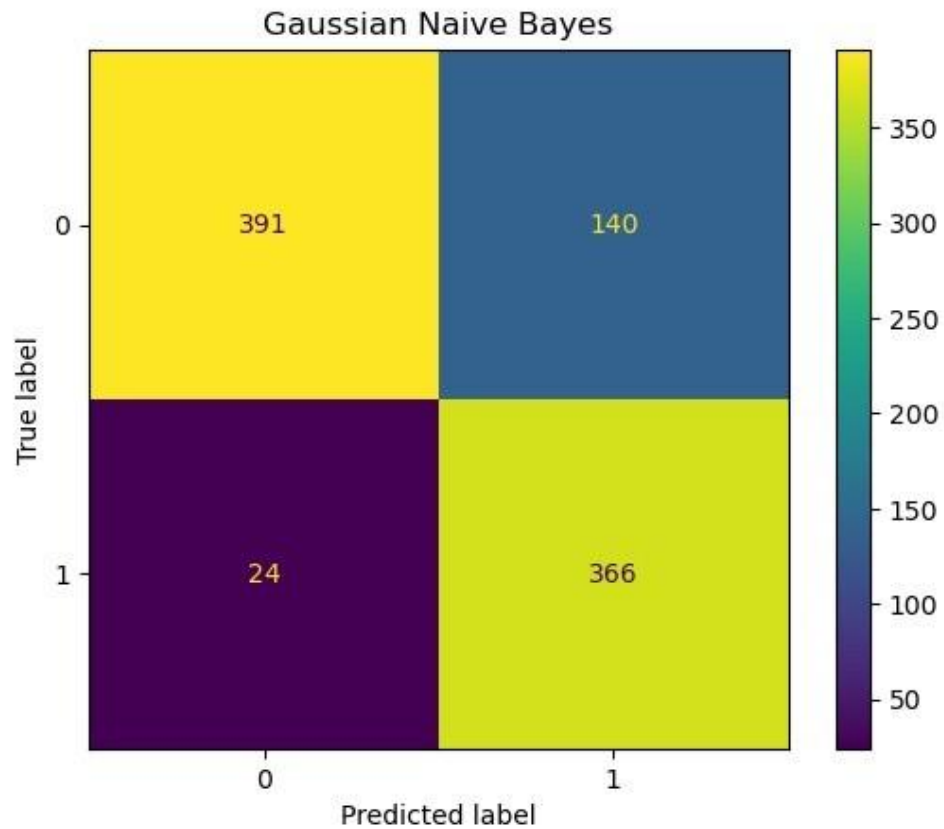
```

	Metric	Gaussian	Multinomial	Bernoulli
0	Accuracy	0.821933		0.871878
		0.880565		
1	Precision	0.723320		0.950331
		0.904624		
2	Recall	0.938462		0.735897
		0.802564		
3	F1	0.816964	0.829480	0.850543
4	ROC-AUC	0.837404		0.853824
		0.870209		

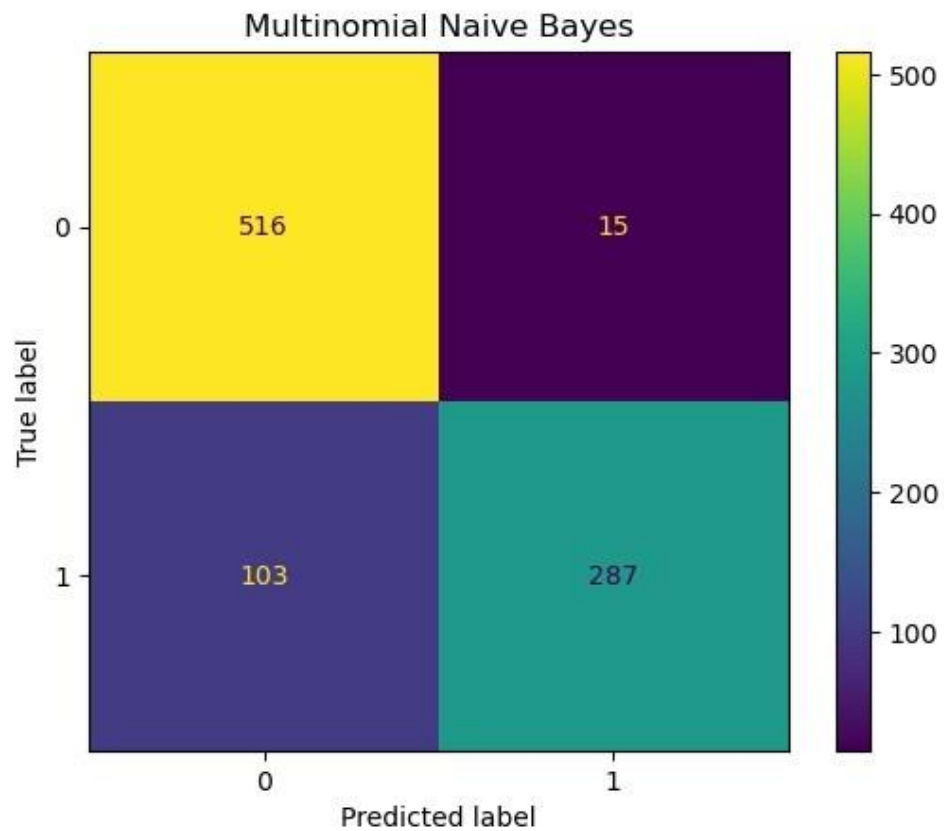
```

ConfusionMatrixDisplay.from_predictions(y_test, y_pred_gnb)
plt.title("Gaussian Naive Bayes") plt.show()

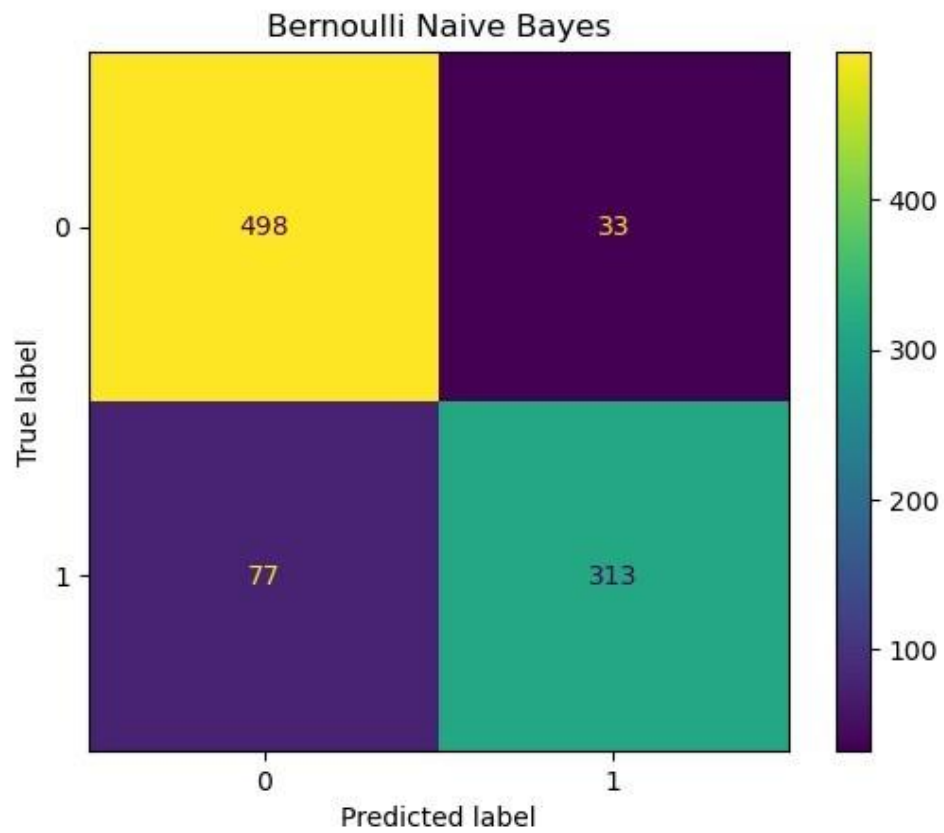
```



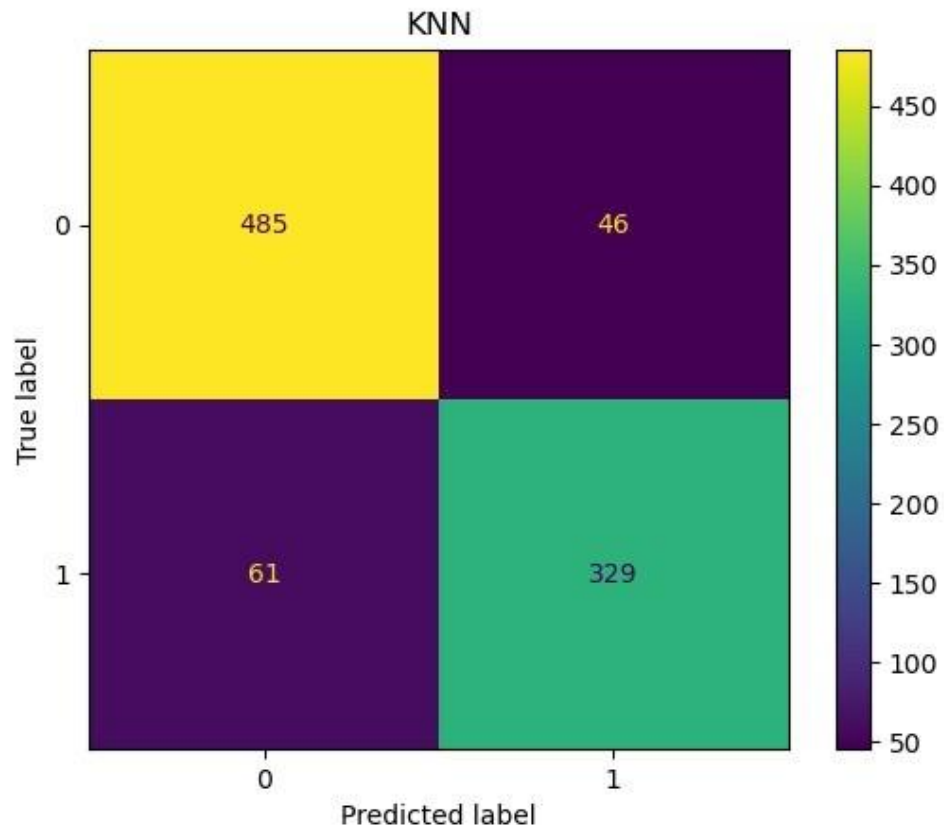
```
ConfusionMatrixDisplay.from_predictions(y_test, y_pred_mnb)  
plt.title("Multinomial Naive Bayes") plt.show()
```



```
ConfusionMatrixDisplay.from_predictions(y_test, y_pred_bnb)
plt.title("Bernoulli Naive Bayes") plt.show()
```



```
ConfusionMatrixDisplay.from_predictions(y_test, y_pred_knn)
plt.title("KNN") plt.show()
```



```
k_values = [1, 3, 5, 7, 9, 11]
results = []
for k in k_values:
    model = KNeighborsClassifier(n_neighbors=k)
    model.fit(X_train, y_train)
    pred = model.predict(X_test)

    accuracy = accuracy_score(y_test, pred)
    precision = precision_score(y_test, pred)
    recall = recall_score(y_test, pred)
    f1 = f1_score(y_test, pred)

    results.append([k, accuracy, precision, recall, f1])

knn_table = pd.DataFrame(
    results,
    columns=["k", "Accuracy", "Precision", "Recall", "F1"]
)
```

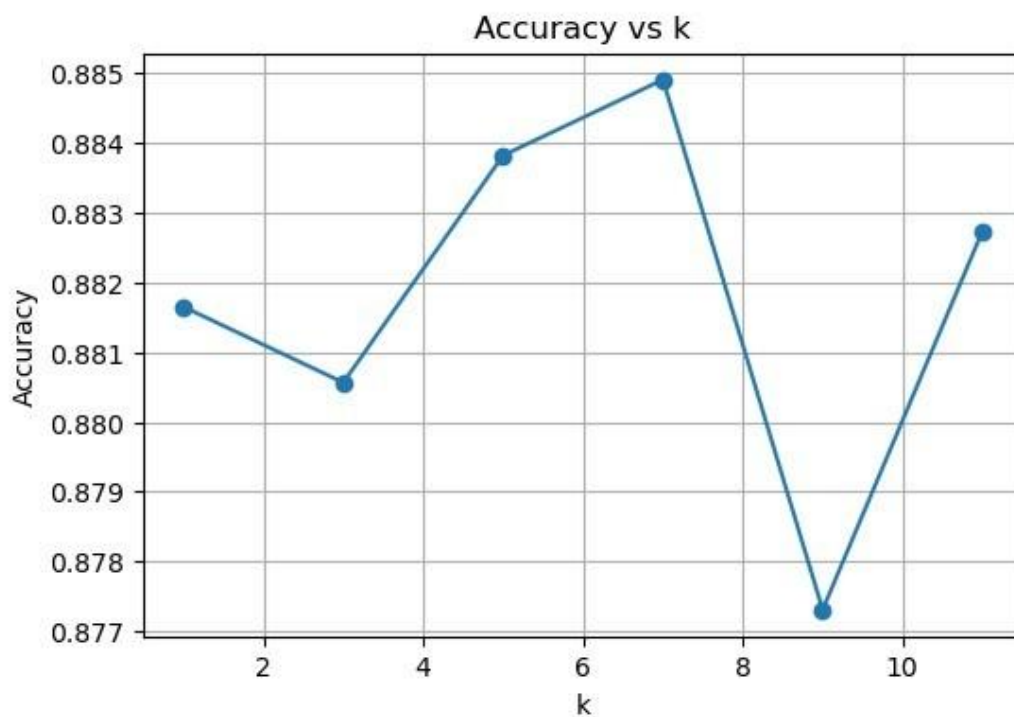
knn_table

k	Accuracy	Precision	Recall	F1	
4	9	0.877307	0.887955	0.812821	0.848728
5	11	0.882736	0.896067	0.817949	0.855228

```
plt.figure(figsize=(6,4))
plt.plot(knn_table["k"],
knn_table["Accuracy"],
marker="o")

plt.title("Accuracy vs k")
plt.xlabel("k")
plt.ylabel("Accuracy")
plt.grid(True)
plt.show()
```

0	1	0.881650	0.850374	0.874359	0.862200
1	3	0.880565	0.872340	0.841026	0.856397
2	5	0.883822	0.877333	0.843590	0.860131
3	7	0.884908	0.879679	0.843590	0.861257



```
parameters = {
    "n_neighbors": [1, 3, 5, 7, 9, 11],
    "weights": ["uniform", "distance"],
    "algorithm": ["auto", "kd_tree", "ball_tree"]
}

start = time.time()

grid_search =
GridSearchCV(      KNeighborsClassifier(),
    parameters,
cv=5 )

grid_search.fit(X_train, y_train)

grid_time = time.time() - start

print("Best Parameters") print(grid_search.best_params_)

print("\nBest CV Score") print(grid_search.best_score_)
Best Parameters
{'algorithm': 'auto', 'n_neighbors': 5, 'weights': 'distance'}
```

Best CV Score

0.9141304347826086

```
random_search =  
RandomizedSearchCV(KNeighborsCl  
assifier(),  
param_distributions=parameters,  
n_iter=10, cv=5,  
random_state=42  
)
```

```
start = time.time()
```

```
random_search.fit(X_train, y_train) random_time
```

```
= time.time() - start
```

```
print("Best Parameters") print(random_search.best_params_)
```

```
print("\nBest CV Score") print(random_search.best_score_)
```

Best Parameters

```
{'weights': 'distance', 'n_neighbors': 5, 'algorithm': 'kd_tree'}
```

Best CV Score

0.9141304347826086

```
search_result = pd.DataFrame({  
    "Parameter": [  
        "Best k",  
        "Weights",  
        "Algorithm",  
        "CV Accuracy",  
        "Execution Time"  
    ],  
    "GridSearchCV": [  
        grid_search.best_params_["n_neighbors"],  
        grid_search.best_params_["weights"],  
        grid_search.best_params_["algorithm"],  
        grid_search.best_score_,  
        grid_time  
    ],  
    "RandomizedSearchCV": [  
        random_search.best_params_["n_neighbors"],  
        random_search.best_params_["weights"],  
        random_search.best_params_["algorithm"],  
        random_search.best_score_,  
        random_time  
    ]  
})
```

```
})
```

```
search_result
```

	Parameter	GridSearchCV	RandomizedSearchCV
0	Best k	5	5
1	Weights		distance
	distance		
2	Algorithm		auto
	kd_tree		
3	CV Accuracy		0.91413
	0.91413		
4	Execution Time		28.398
	9.394		

```
algorithms = ["kd_tree", "ball_tree"]
```

```
tree_result = [] for algo in
```

```
algorithms:
```

```
    model =  
    KNeighborsClassifier(n_neigh  
bors=5, algorithm=algo  
    )
```

```
    start = time.time()  
    model.fit(X_train, y_train)    train_time  
    = time.time() - start
```

```
    start = time.time()    pred =  
    model.predict(X_test)    predict_time =  
    time.time() - start    accuracy =  
    accuracy_score(y_test, pred)
```

```
    tree_result.append(  
        [algo, accuracy, train_time, predict_time]  
    )
```

```
tree_table =  
pd.DataFrame(  
    tree_resu  
lt,  
columns=[  
    "Algorith  
m",  
    "Accuracy",  
    "Training Time",  
    "Prediction Time"  
    ]  
)
```

```
tree_table
```

	Algorithm	Accuracy	Training Time	Prediction Time
0	kd_tree	0.883822	0.041	0.264
1	ball_tree	0.883822		0.020
		0.229		

```

nb_cv = cross_val_score(GaussianNB(), X, y, cv=5)

best_knn = KNeighborsClassifier(
    n_neighbors=grid_search.best_params_["n_neighbors"],
    weights=grid_search.best_params_["weights"],
    algorithm=grid_search.best_params_["algorithm"]
)

knn_cv = cross_val_score(best_knn, X, y, cv=5)

cv_table =
pd.DataFrame({
    "Fold": [1, 2, 3, 4, 5],
    "Naive Bayes": nb_cv,
    "Best KNN": knn_cv
})
cv_table

```

	Fold	Naive Bayes	Best KNN
0	1	0.851249	0.876221
1	2	0.866304	0.892391
2	3	0.854348	0.918478
3	4	0.843478	0.909783
4	5	0.695652	0.763043

```

average = pd.DataFrame({
    "Model": ["Naive Bayes", "Best KNN"],
    "Average
Accuracy": [
        nb_cv.mean(),
        knn_cv.mean()
    ]
})
average

```

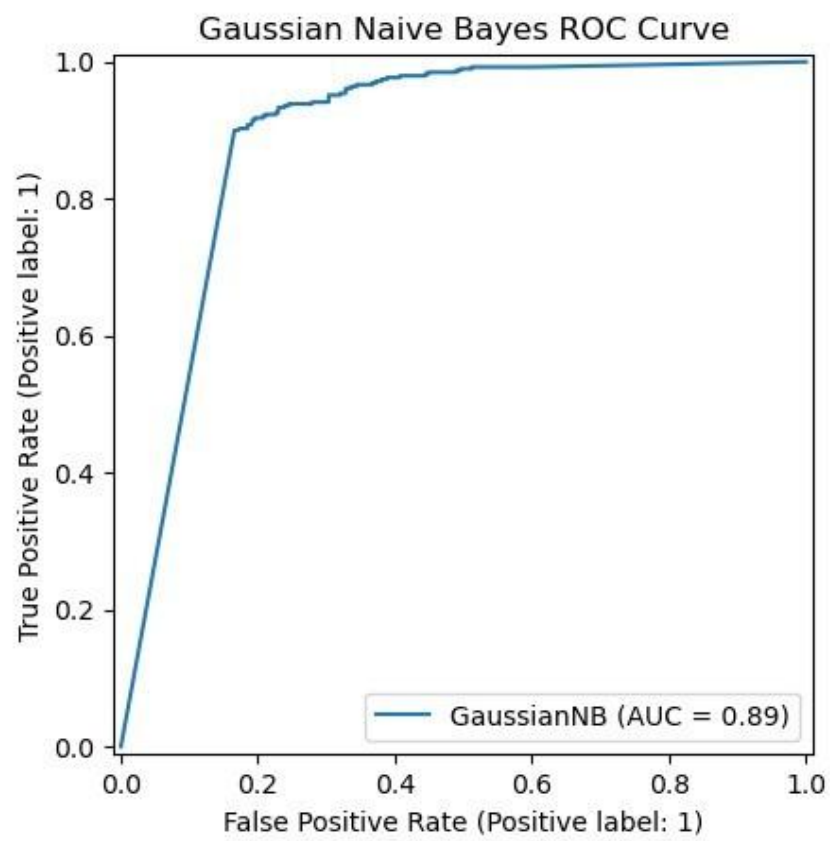
	Model	Average Accuracy
0	Naive Bayes	0.822206
1	Best KNN	0.871983

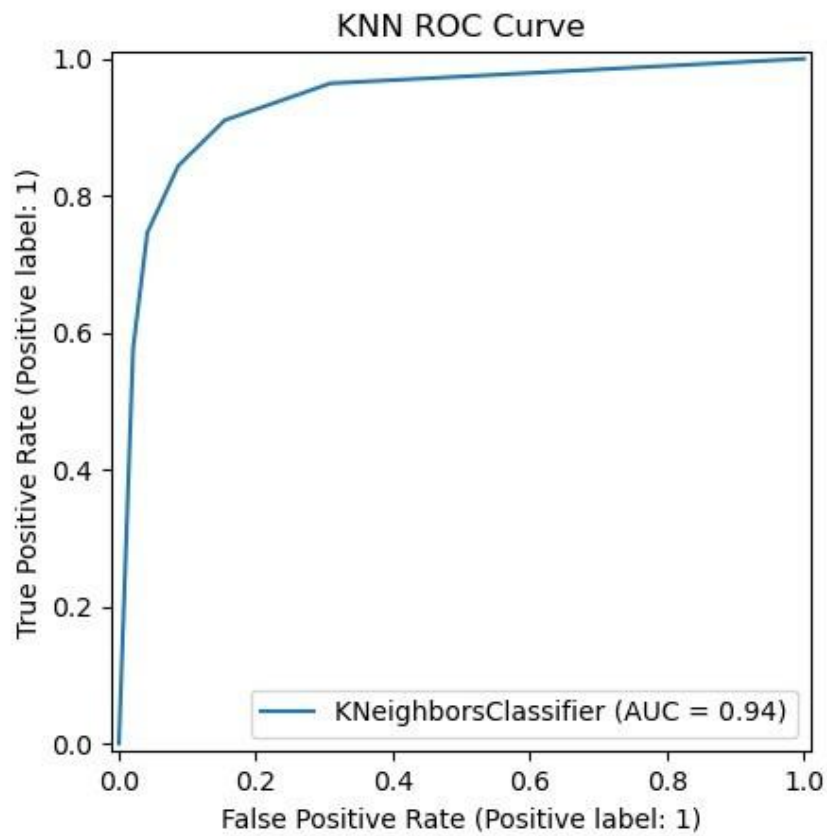
```

RocCurveDisplay.from_estimator(gnb, X_test, y_test)
plt.title("Gaussian Naive Bayes ROC Curve") plt.show()

RocCurveDisplay.from_estimator(knn, X_test, y_test)
plt.title("KNN ROC Curve") plt.show()

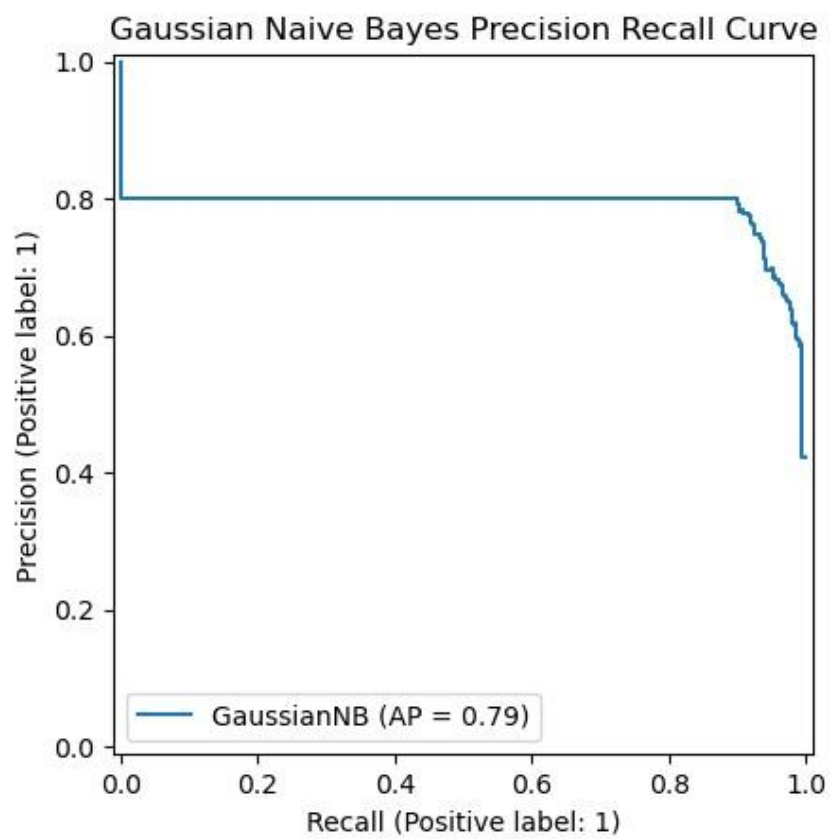
```

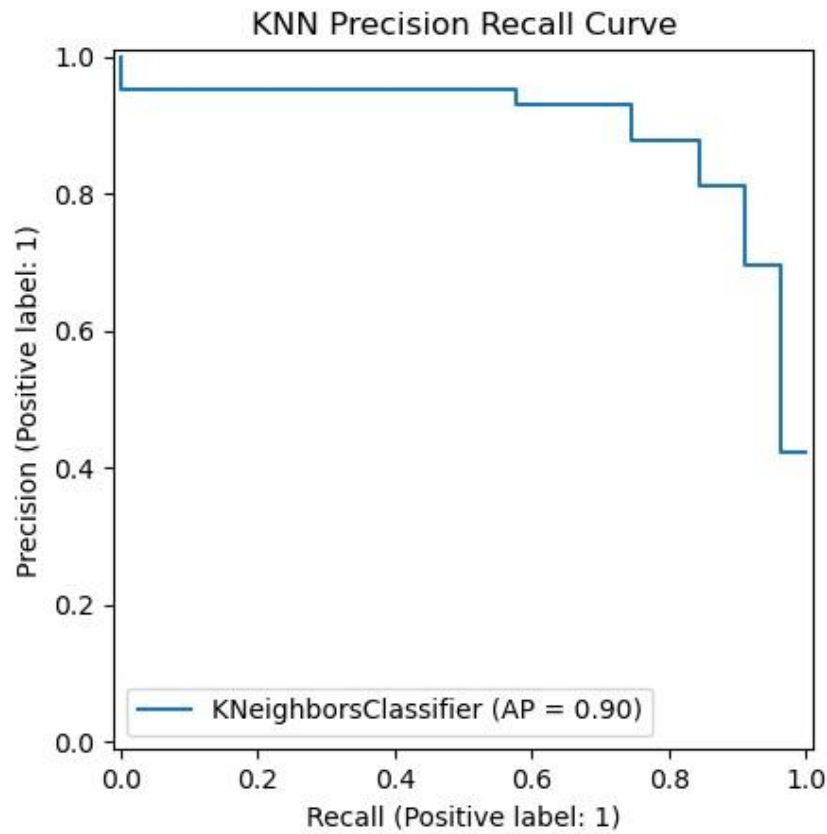




```
PrecisionRecallDisplay.from_estimator(gnb, X_test, y_test)
plt.title("Gaussian Naive Bayes Precision Recall Curve")
plt.show()
```

```
PrecisionRecallDisplay.from_estimator(knn, X_test, y_test)
plt.title("KNN Precision Recall Curve") plt.show()
```





```
training =
pd.DataFrame({      "Algorithm":[
    "Gaussian NB",
    "Multinomial NB",
    "Bernoulli NB",
    "Best KNN"
],
    "Training
Time":[      train_time_gnb,
train_time_mnb,
train_time_bnb,
train_time_knn
]
})
```

```
training
```

	Algorithm	Training Time
0	Gaussian	NB 0.008
1	Multinomial	NB 0.007
2	Bernoulli	NB 0.012
3	Best KNN	0.002

```

prediction = pd.DataFrame({
    "Algorithm":[
        "Gaussian NB",
        "Multinomial NB",
        "Bernoulli NB",
        "Best KNN"
    ],
    "Prediction
Time":[
        prediction_time_gnb,
prediction_time_mnb,
prediction_time_bnb,
prediction_time_knn
    ]
})
prediction

```

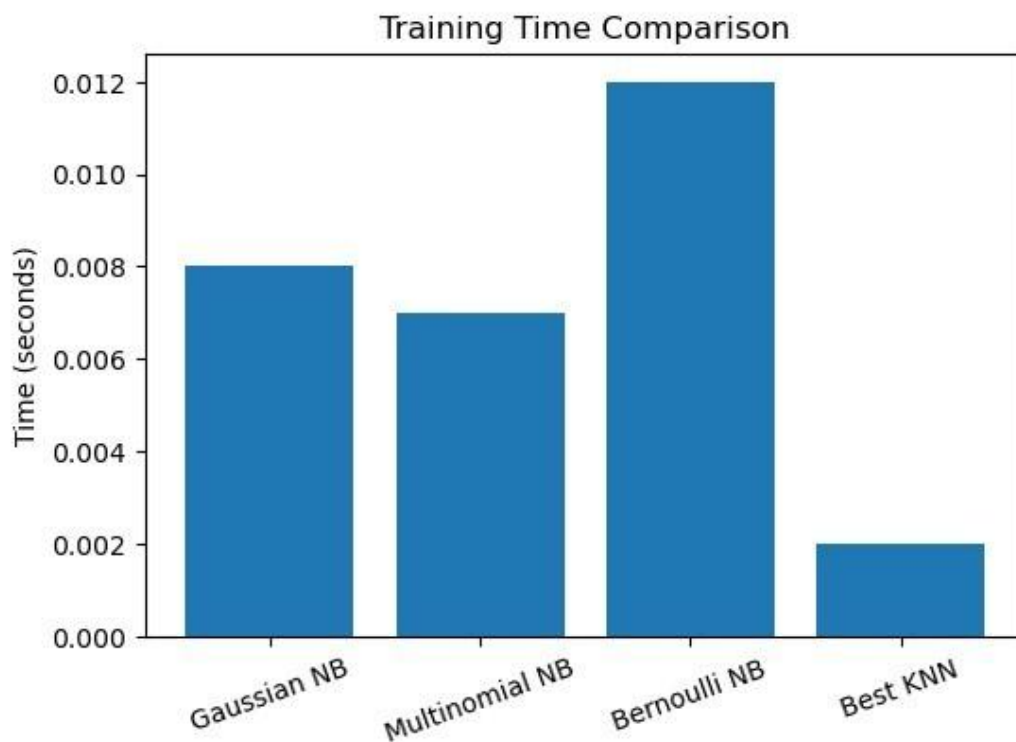
	Algorithm	Prediction Time
0	Gaussian	NB 0.002
1	Multinomial	NB 0.001
2	Bernoulli	NB 0.002
3	Best KNN	0.111

```

plt.figure(figsize=(6,4))

plt.bar(training["Algorithm"], training["Training Time"])
plt.title("Training Time Comparison") plt.ylabel("Time
(seconds)") plt.xticks(rotation=20) plt.show()

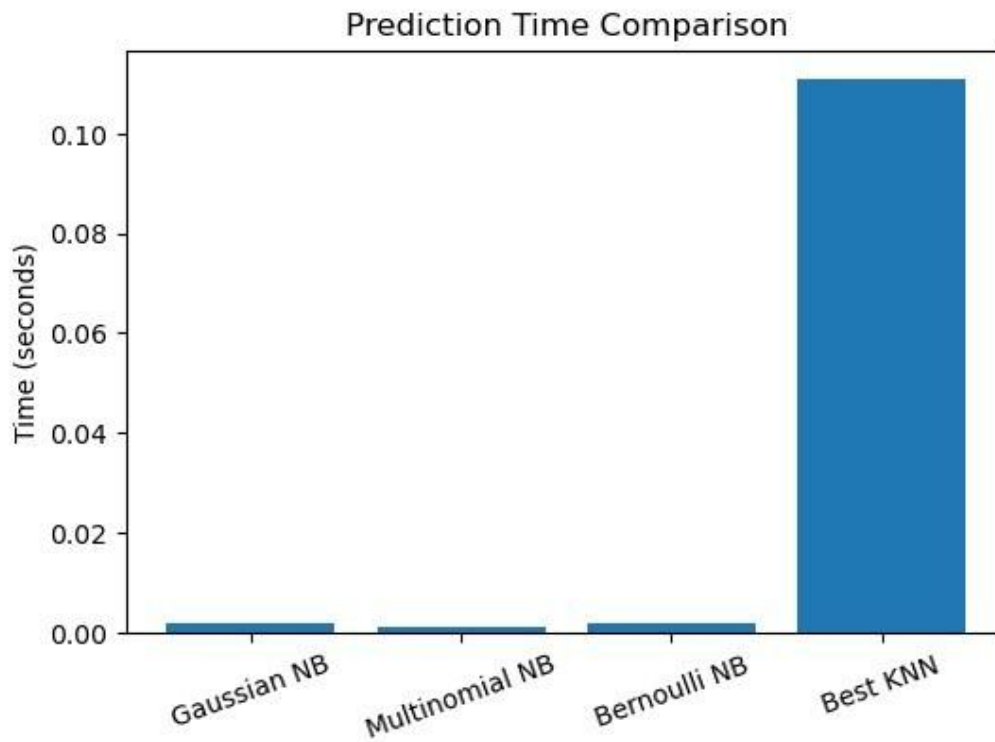
```



```
plt.figure(figsize=(6,4))

plt.bar(prediction["Algorithm"], prediction["Prediction Time"])

plt.title("Prediction Time Comparison") plt.ylabel("Time
(seconds)") plt.xticks(rotation=20) plt.show()
```



```
classifier = pd.DataFrame({
    "Classifier":[
        "Gaussian NB",
        "Multinomial NB",
        "Bernoulli NB",
        "KNN"
    ],

    "Accuracy":[
        g[0],
        m[0],          b[0],
        evaluate(y_test, y_pred_knn)[0]
    ]
})
classifier

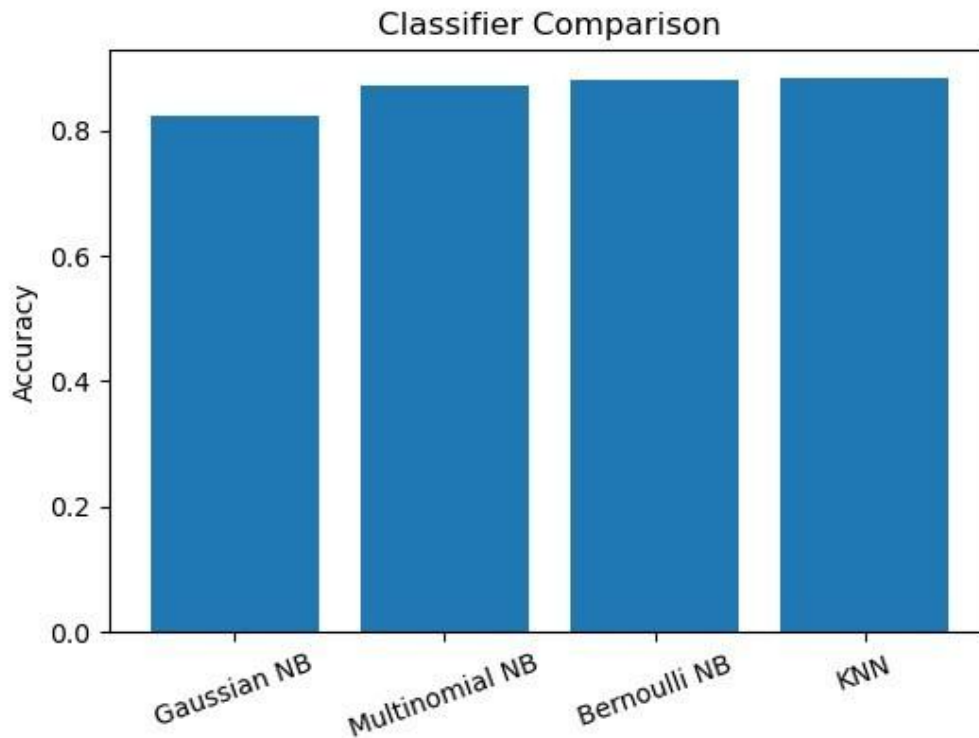
```

	Classifier	Accuracy
0	Gaussian NB	0.821933
1	Multinomial NB	0.871878
2	Bernoulli NB	0.880565
3	KNN	0.883822

```
plt.figure(figsize=(6,4))

plt.bar(classifier["Classifier"], classifier["Accuracy"])
```

```
plt.title("Classifier Comparison")
plt.ylabel("Accuracy") plt.xticks(rotation=20)
plt.show()
```



Hyperparameter Tuning

GridSearchCV and RandomizedSearchCV were used to find suitable parameters for the KNN classifier. Different values of k , weights and algorithms were tested and the best combination was selected based on cross-validation accuracy.

Comparison Tables

Comparison tables for Naïve Bayes models, KNN models, GridSearchCV, RandomizedSearchCV, KDTree, BallTree and Cross Validation are included in the attached notebook.

Complexity Analysis

Algorithm	Training	Prediction
Naïve Bayes	$O(nd)$	$O(d)$
KNN	$O(1)$	$O(nd)$
KDTree	$O(n \log n)$	$O(\log n)$
BallTree	$O(n \log n)$	$O(\log n)$

Plots

The experiment includes the following plots.

- Class distribution
- Correlation heatmap
- Histograms
- Boxplots
- Confusion matrices
- ROC curves
- Precision-Recall curves
- Accuracy vs k
- Cross-validation accuracy
- Training time comparison
- Prediction time comparison
- Classifier comparison bar chart

Results

The models were trained and evaluated successfully. Their performance was compared using accuracy, precision, recall, F1-score and ROC-AUC. The comparison helped identify the better model for email spam classification.

Conclusion

In this experiment, Email Spam/Ham classification was performed using Naïve Bayes and KNN. The performance of different models was compared using various evaluation metrics. Hyperparameter tuning and cross validation helped in selecting a suitable model for the given dataset.

